Digital VLSI Design

4-bit Array Multiplier from RTL to GDSII

Assignment Report

Team member	**Roll number**
Gaurav Reddy Yerreddi	2024112006
Yogesh Krishna	2024102035

Item	**Detail**
Design	4-bit × 4-bit unsigned array multiplier (8-bit product)
Technology	SkyWater SKY130 (130 nm), `sky130_fd_sc_hd`
Tools	Icarus Verilog, GTKWave, Yosys, OpenSTA, LibreLane, KLayout
Repository	`gauravv-13/mult4-array-multiplier`

1. Part A – RTL Design and Verification

1.1 Full adder (`rtl/full_adder.v`)

Module `full_adder` has inputs `a`, `b`, `cin` and outputs `sum`, `cout`. Both outputs are written as continuous `assign` statements: $sum = a \oplus b \oplus cin$ and $cout = ab + cin\,(a \oplus b)$.

1.2 Array multiplier (`rtl/mult_array.v`)

On every rising edge of `clk`, inputs `a` and `b` are captured into the 4-bit registers `a_r` and `b_r`. Sixteen AND gates form the partial products $pp_i[j] = a_r[j] \cdot b_r[i]$ (`pp0`–`pp3`). Row 0 is AND gates only: $p[0] = pp_0[0]$, and $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$ is passed down. Rows 1–3 are each a 4-bit ripple-carry adder of four `full_adder` instances (12 in total, `fa1_0`–`fa3_3`), wired exactly as in the connection table in Section 2 of the assignment handout. Each row's lowest sum bit gives $p[1]$, $p[2]$, $p[3]$, and row 3 also gives $p[7{:}4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$. The 8-bit result is registered into `p`. The design therefore has 16 flip-flops (4 + 4 + 8), all cleared by the asynchronous active-low reset `rst_n`, and a latency of 2 clock cycles. The `*` operator is not used.

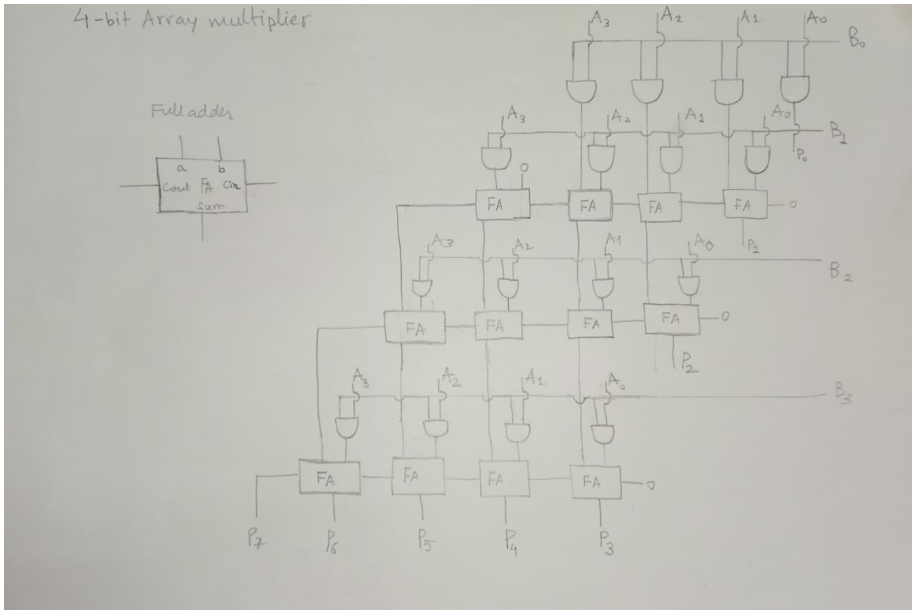


Figure 1: Block diagram of the 4-bit array multiplier, A[3:0]*B[3:0] = P[7:0]

1.3 Testbench (`tb/tb_mult.v`)

The testbench generates a 10 ns clock (`#5` half period) and holds `rst_n` low from 0 to 26 ns, which covers the first rising edges at 5, 15 and 25 ns. Two nested loops apply all 256 combinations ($a, b = 0 \ldots 15$). For each vector the testbench drives `a` and `b`, waits for two rising edges (the 2-cycle latency) plus 2 ns, and compares `p` with `a * b` (the `*` operator is used only in the testbench). The next vector is driven immediately after the check, so every input change happens 2 ns after a rising edge.

The 2 ns delay places the check after the 1 ns clock-to-output delay that the SKY130 flip-flop models have in gate-level simulation (`UNIT_DELAY=#1`). With a 1 ns delay, the check raced the flip-flop update and read the previous product.

Every mismatch is printed, a single `PASS/FAIL` summary line is printed at the end, and the waveform is written to `mult.vcd`.

1.4 RTL simulation result

```
$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
$ vvp sim
PASS: 256 vectors checked, 0 errors
```

1.5 Hand-worked example and waveform

Example chosen using the roll number of Gaurav Reddy Yerreddi (2024112006): last digit 6, so $A = 6 + 5 = 11$ and $B = 7$.

```
        1 0 1 1      A = 11
      x 0 1 1 1      B = 7
  ---------------
        1 0 1 1      pp0 = A AND b0 (b0 = 1)
      1 0 1 1        pp1 = A AND b1 (b1 = 1), shifted 1
    1 0 1 1          pp2 = A AND b2 (b2 = 1), shifted 2
  0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
  ---------------
0 1 0 0 1 1 0 1      P = 77
```

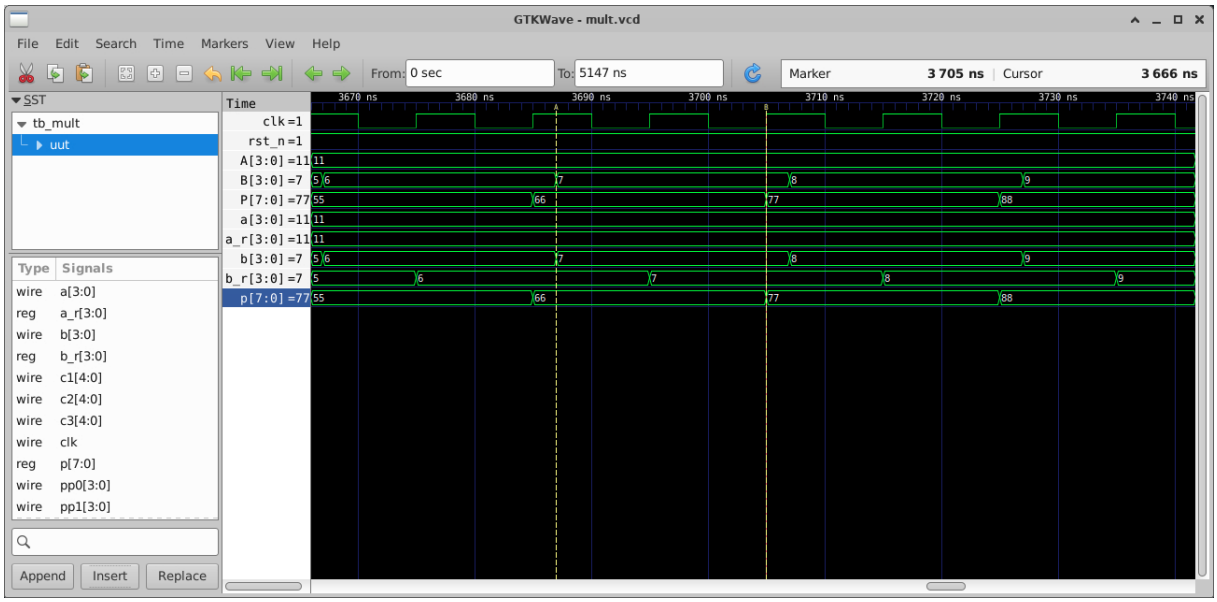


Figure 2: Waveform for $A = 11$, $B = 7$. Marker A ($t = 3687$ ns): b changes to 7, 2 ns after a rising edge. Marker B ($t = 3705$ ns): $p = 77$ appears at the second rising edge after the inputs are applied.

The clock has rising edges at 3685, 3695 and 3705 ns:

- **3687 ns (marker A):** the testbench changes b from 6 to 7, 2 ns after the edge at 3685 ns.
- **3695 ns (edge 1):** $a = 11$ and $b = 7$ are captured into `a_r` and `b_r`, and the array computes the product.
- **3705 ns (edge 2, marker B):** the product is registered and `p` changes from 66 to 77, confirming the 2-cycle latency.

The 2 ns hold margin keeps the input from changing at the same instant as a clock edge, so the edge at 3685 ns cleanly samples the old value. Without it the RTL simulation still passes, because its registers update with zero delay, but the synthesized netlist fails: its flip-flop models have a 1 ns clock-to-output delay, so the check and the input change race the flip-flop update.

1.6 Error-detection test

With `sum` in `full_adder.v` temporarily changed to `a ^ b`, the testbench prints:

```
FAIL: 256 vectors checked, 95 errors
```

The change was then undone and the testbench again printed PASS.

2. Part B – Logic Synthesis with Yosys

2.1 Synthesis script

The design was synthesized with Yosys into a single flattened module, merging the twelve full adders into the top level. The registers and logic were then mapped to SKY130 standard cells using the typical-corner library, and the resulting netlist and area report were written out.

2.2 Results

Metric	Value
Total number of cells	68
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	785.754
Flip-flop area (μm^2)	400.384
Flip-flop area (% of total)	50.96 %
The 3 most-used cell types and their counts	• Flip-flops: 16 • 2-input NAND gates: 8 • 2-input AND and XNOR gates: 6 each (tied for third)
Netlist simulation result (PASS/FAIL line)	`PASS: 256 vectors checked, 0 errors`

Flip-flop area = 400.384 μm^2, which is 400.384/785.754 = 50.96 % of the total.
Area of one flip-flop = 400.384/16 = 25.024 μm^2.

2.3 Gate-level simulation

```
$ CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
$ iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls tb/tb_mult.v \
    $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
$ vvp gls
PASS: 256 vectors checked, 0 errors
```

3. Part C – Static Timing Analysis with OpenSTA

3.1 Minimum period and maximum frequency

Run 1, $T = 10\,\mathbf{ns}$ (`set T 10.0` in `sta/sta.tcl`):

```
$ sta sta/sta.tcl
worst slack max 7.03

mult_array_timing.rpt (clk path group):
            7.027   slack (MET)
```

$$T_{\min} = T - \mathrm{WNS} = 10 - 7.027 = 2.973 \text{ ns}, \qquad F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.973} = 336.4 \text{ MHz}$$

Run 2, $T = T_{\min} = 2.973\,\mathbf{ns}$ (`set T 2.973`):

```
$ sta sta/sta.tcl
worst slack max -0.00

mult_array_timing.rpt (clk path group):
            0.000   slack (VIOLATED)
```

The slack is zero to three decimal places, which confirms $T_{\min}$. It is flagged as violated only because 7.027 is a rounded value, so 2.973 ns is a fraction of a picosecond shorter than the true minimum period.

3.2 Critical path

The critical path is the slowest register-to-register path, and it sets $T_{\min}$.

- **Start point:** input register `b_r[0]` (flip-flop `_108_`).
- **End point:** output register `p[7]` (flip-flop `_103_`), driven by the final carry $c_3[4]$.

The path crosses the whole array: $b[0]$ feeds the first adder stage, and $p[7]$ settles last because it waits for the carry to ripple through all three rows.

#	Instance	Cell type	From a full adder?	Delay (ns)	Arrival (ns)
1	_108_	dfrtp_1 (CLK→Q, b_r[0])	No (input register)	0.410	0.410
2	_050_	and4_1	No (partial-product AND gates)	0.223	0.634
3	_061_	o21ai_0	Yes (row-1 carry logic)	0.195	0.828
4	_062_	nand3_1	Yes (row-1 carry logic)	0.140	0.969
5	_063_	maj3_1	Yes (carry)	0.380	1.349
6	_069_	xnor3_1	Yes (sum)	0.393	1.741
7	_074_	maj3_1	Yes (carry)	0.369	2.110
8	_086_	maj3_1	Yes (carry)	0.383	2.493
9	_087_	maj3_1	Yes (carry, $c_3[4]$)	0.349	2.842
10	_103_	dfrtp_1 (D, p[7])	No (output register; setup 0.131)	0.000	2.842

After the 0.410 ns clock-to-Q delay, cells 2–4 form the partial products and the first row-1 carry, and cells 5–9 are the carry ripple down to $c_3[4]$, which arrives at 2.842 ns. With a 0.131 ns setup time, the data is required at $2.973 - 0.131 = 2.842$ ns, so the slack at $T_{\min}$ is zero.

3.3 Comparison with the full-adder estimate

7 of the 8 combinational cells on the path come from full adders (cells 3–9), compared with the estimate of about $2N = 8$ full-adder delays. The path is slightly shorter for three reasons:

1. Synthesis merged the AND gates into the first adder stage.
2. Full adders with a constant-0 input ($c_i[0] = 0$, $r_0[3] = 0$) simplify into cheaper logic.
3. A full adder on the carry chain contributes only its carry, which a single maj3 cell computes in one cell delay.

3.4 Results

Metric	**Value**
Worst slack at $T = 10\,\text{ns}$ (ns)	7.027
T_{min} (ns)	2.973
F_{max} (MHz)	336.4
Slack at $T = T_{\text{min}}$ (ns)	0.000
Critical path start point	`b_r[0]` (flip-flop `_108_`)
Critical path end point	`p[7]` (flip-flop `_103_`)
Number of cells on the critical path	10 (launch flip-flop + 8 combinational cells + capture flip-flop)
Latency (ns) $= 2 \times T_{\text{min}}$	5.946
Throughput (million results per second)	336.4

4. Part D – Physical Design: RTL to GDSII

4.1 Configuration

`CLOCK_PERIOD` $= 1.2 \times T_{\min} = 1.2 \times 2.973 = 3.568$ ns, rounded up to the next 0.5 ns = 4.0 ns. This was the starting value. The first run failed setup timing, so, as the handout instructs, the period was increased in 0.5 ns steps until the flow passed at 6.0 ns (Section 4.2).

Key	**Value used**
`DESIGN_NAME`	`"mult_array"`
`VERILOG_FILES`	`["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"]`
`CLOCK_PORT`	`"clk"`
`CLOCK_PERIOD`	`6.0` (final run; first run 4.0)
`FP_SIZING`	`"absolute"`
`DIE_AREA`	`[0, 0, 150, 150]`

4.2 Runs

Run	`CLOCK_PERIOD` **(ns)**	**Setup slack (ns)**	**Outcome (DRC / LVS / timing)**
1	4.0	−1.688	0 / 0 / setup violated (slow corner `ss_100C_1v60`; typical corner +1.011)
2	4.5	−1.188	0 / 0 / setup violated (slow corner)
3	5.0	−0.688	0 / 0 / setup violated (slow corner)
4	5.5	−0.188	0 / 0 / setup violated (slow corner)
5	6.0	+0.046	0 / 0 / **timing met in all corners** (typical corner +2.916)

LibreLane signs off setup timing in every process corner: typical (`tt_025C_1v80`), slow (`ss_100C_1v60`) and fast (`ff_n40C_1v95`). The reported `timing__setup__ws` is the worst of these, which is always the slow corner (1.60 V, 100 °C). Runs 1–4 met timing in the typical corner but failed in the slow corner.

4.3 Results

Metric	Value
`CLOCK_PERIOD` used (ns)	6.0
Number of runs needed	5
Core area (μm^2)	17 759.5
Cells after synthesis (from Part B)	68
Cells after place and route	1 891
Kinds of cells added	Decap (1 300), fill (252), tap (245), buffers (26: 23 timing-repair + 3 clock-tree)
Wirelength (μm)	2 407
Setup slack (ns)	+0.046 (worst corner, `ss_100C_1v60`)
Post-route $F_{\max}$ (MHz)	167.9
$F_{\max}$ from Part C (MHz)	336.4
DRC errors	0 (Magic and KLayout)
LVS errors	0

4.4 Post-route maximum frequency

$$F_{\max,\text{post-route}} = \frac{1000}{\texttt{CLOCK_PERIOD} - \text{setup slack}} = \frac{1000}{6.0 - 0.046} = 167.9 \text{ MHz}$$

Compared with $F_{\max} = 336.4$ MHz from Part C, the post-route $F_{\max}$ is about half. Most of this gap comes from the corner: Part C analysed only the typical corner, while the post-route slack comes from the slow corner. In the typical corner alone, the post-route slack is +2.916 ns, which gives $1000/(6.0 - 2.916) = 324.2$ MHz, only 3.6 % below Part C. That remaining difference is the cost of real wires and a real clock tree (Q4).

4.5 Cells added by the flow

Cells after synthesis: 68; after place and route: 1 891; added: 1 823. The 68 logic cells (16 `dfrtp_1` flip-flops, 4 inverters, 48 other gates) are still present. All the added cells are of the four kinds below (counted from the final GDS). `metrics.json` reports decap and fill cells together as one fill-cell count of 1 552.

Kind	Example cell name	Count	Purpose
Buffer	`buf_2`, `buf_4`, `clkbuf_16`, `clkdlybuf4s50_1`	26	Clock-tree buffers (3 `clkbuf_16`) distribute the clock to all 16 flip-flops with low skew. Timing-repair buffers (23) drive long wires and high fan-out nets and fix slew and setup problems.
Fill	`fill_1`, `fill_2`	252	Fill the empty gaps in the cell rows so that the wells, implants and power rails are continuous, as the DRC rules require. They contain no logic.
Tap	`tapvpwrvgnd_1`	245	Tie the n-well to VPWR and the p-substrate to VGND at regular intervals. This prevents latch-up.
Decap	`decap_3/4/6/8/12`	1 300	On-chip capacitors between VPWR and VGND. They supply the current spikes drawn when gates switch, reducing supply noise (IR drop). They also use up free space in the rows.

4.6 Layout

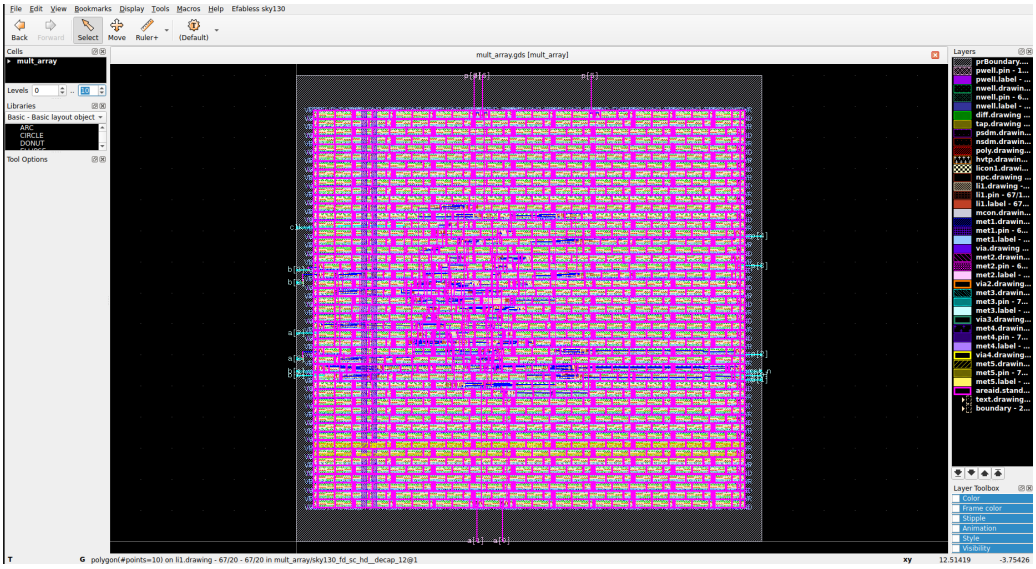


Figure 3: Final layout, whole 150 μm × 150 μm die.

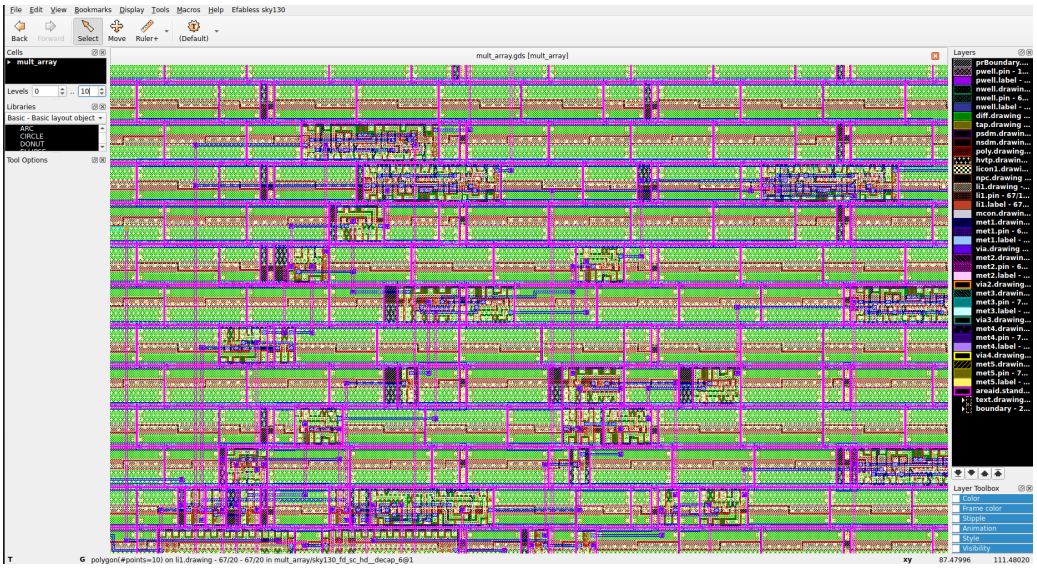


Figure 4: Zoomed view showing standard cell rows and metal routing.

5. Answers to Questions

Q1. Logic cell count

> *The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic (total cells minus flip-flops)? Explain why the number of cells is not simply* $16 + 12$.

Synthesis produced $68 - 16 = 52$ logic cells, not $16 + 12 = 28$.

The `sky130_fd_sc_hd` library has no single full-adder cell that ABC used here. Each full adder is split into a sum function (built from `xnor2`, `xnor3` and `xor3` cells) and a carry function (a `maj3` cell, or NAND/AOI/OAI combinations such as `a21oi`, `o21ai`, `a22oi`), so one adder can map to two or more cells.

ABC also restructures across the flattened design. It merges AND gates into wider cells (`and4`, `nand4`, `a2111oi`), shares logic between adders, and simplifies adders whose inputs are constant 0, for example $c_i[0] = 0$ and $r_0[3] = 0$. It also uses cells that the RTL does not name, such as 2 `clkinv` inverters and 2 `lpflow_isobufsrc` cells, which compute $A \cdot \overline{B}$ and serve here as ordinary logic gates.

The count is therefore set by how the logic decomposes into library cells, not by the number of AND gates and full adders in the RTL.

Q2. Critical path

> *What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about* $2N$ *full-adder delays?*

The critical path runs from input register `b_r[0]` (flip-flop `_108_`) to output register `p[7]` (flip-flop `_103_`), whose D input is the final carry $c_3[4]$. It has 10 cells: the launch flip-flop (0.410 ns clock-to-Q), 8 combinational cells (`and4`, `o21ai`, `nand3`, `maj3`, `xnor3`, `maj3`, `maj3`, `maj3`), and the capture flip-flop (0.131 ns setup). The data arrival time is 2.842 ns, which gives $T_{\min} = 2.973$ ns.

Seven of the 8 logic cells come from full adders, so the path is about 7 full-adder stages against the $2N = 8$ estimate. This matches the expected shape: the path ripples through the carry chains and down the rows to the most significant product bit. It is slightly shorter because ABC merged the partial-product AND gates and the first carries into compound cells, simplified adders with constant-0 inputs, and implemented most carry-chain stages as a single `maj3` cell.

Q3. Flip-flop area

> *What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?*

Flip-flops take 400.384/785.754 = 50.96 % of the total cell area after synthesis, although they are only 16/68 = 23.5 % of the cells. A `dfrtp_1` cell (25.02 μm^2) holds two latches, clock buffering and the asynchronous-reset logic, so it is about 2.5–7 times larger than a typical logic gate here (`nand2_1` is 3.75 μm^2, `maj3_1` is 10.0 μm^2).

The register count is fixed by the interface: 8 input bits and 8 output bits always need 16 flip-flops. The combinational logic of a 4-bit multiplier is very small, so the registers dominate. For an N-bit multiplier the logic grows as N^2 but the registers only as $4N$, so the flip-flop percentage falls as the design gets wider.

Q4. Post-route vs. pre-route $F_{\max}$

> *Why is the post-route $F_{\max}$ lower than the $F_{\max}$ from Part C? Which kinds of cells did the flow add, and why is each kind needed?*

The post-route $F_{\max}$ is 167.9 MHz, compared with 336.4 MHz from Part C, for three reasons.

1. Part C used ideal wires: OpenSTA saw only cell delays, with zero wire resistance and capacitance. After routing, the extracted RC of 2 407 μm of metal adds delay to every net and slows the input transitions of every cell.
2. Part C used an ideal clock that reached all flip-flops at the same instant (`clock network delay (ideal)`). After clock tree synthesis the clock passes through real buffers, and the flow reports about 0.26 ns of clock skew in the worst setup case, which can take time away from a path.
3. Most importantly, LibreLane signs off at the slow corner (`ss_100C_1v60`: slow transistors, 1.60 V, 100 °C), while Part C used only the typical corner (`tt_025C_1v80`). Gates are much slower in the slow corner, so this corner sets the worst slack (+0.046 ns at 6.0 ns).

In the typical corner alone, post-route $F_{\max} = 1000/(6.0 - 2.916) = 324.2$ MHz, which is only 3.6 % below Part C. That small gap is the cost of effects (1) and (2).

The flow added 1 823 cells to the 68 logic cells:

- **Buffers (26):** 3 clock-tree buffers (`clkbuf_16`) to deliver the clock with low skew, and 23 timing-repair buffers (`buf_2`, `buf_4`, `clkdlybuf4s50_1`) to drive long or heavily loaded nets and fix slew and setup problems.
- **Tap cells (245):** (`tapvpwrvgnd_1`) tie the n-well to VPWR and the p-substrate to VGND at regular intervals to prevent latch-up.
- **Decap cells (1 300):** (`decap_*`) act as local charge reservoirs between VPWR and VGND that reduce supply noise when gates switch.
- **Fill cells (252):** (`fill_*`) close the remaining empty gaps in the standard cell rows so that the wells, implants, and power rails are continuous and the layout passes DRC.

Q5. DRC and LVS

What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

DRC (Design Rule Check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths and spacings of each layer, enclosure of vias and contacts, minimum area, density, and similar rules. A violation means the foundry cannot reliably fabricate that shape, so wires may short together or break open.

LVS (Layout Versus Schematic) extracts the transistors and their connections from the layout and compares them with the gate-level netlist. It confirms that the layout implements exactly the intended circuit, with no missing or extra devices, opens or shorts.

Both must pass because they catch different failures. A layout can be DRC-clean but wired wrongly (it fails LVS), or correctly connected but impossible to manufacture (it fails DRC). Either way the fabricated chip would not work, and a mask set is expensive. Our run reports 0 DRC errors (Magic and KLayout) and 0 LVS errors.

Q6. Scaling to 8 bits

Using the formulas in Section 2 of the assignment handout, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

Full adders: $N(N-1)$ gives $4 \times 3 = 12$ for $N = 4$ and $8 \times 7 = 56$ for $N = 8$, which is $56/12 \approx 4.7\times$ more. AND gates grow from $N^2 = 16$ to 64 ($4\times$).

Critical path: about $2N$ full-adder delays, so 8 for $N = 4$ and 16 for $N = 8$, about twice as long. From our STA, the combinational part of the path is $2.842 - 0.410 = 2.432$ ns (clock-to-Q excluded). Doubling it gives about 4.86 ns. Adding clock-to-Q (0.410 ns) and setup (0.131 ns) gives

$$T_{\min,8} \approx 0.410 + 2 \times 2.432 + 0.131 \approx 5.41 \text{ ns}, \qquad F_{\max,8} \approx \frac{1000}{5.41} \approx 185 \text{ MHz}$$

compared with 2.973 ns and 336.4 MHz for $N = 4$. The hardware grows quadratically, while the delay grows only linearly, about $2\times$.